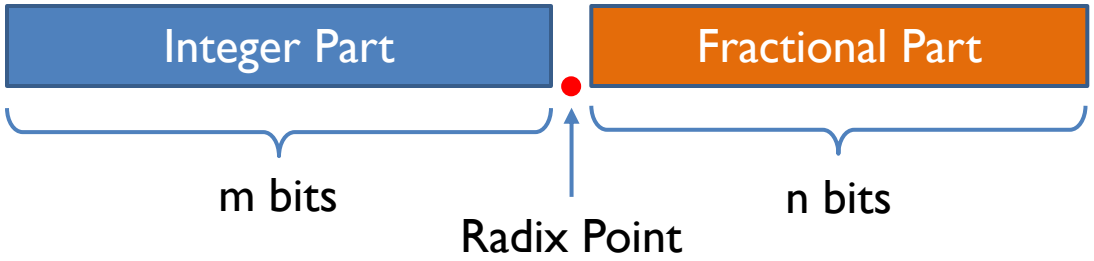
Fixed-Point Numbers

CSE 4095
Special Topics in Computer Engineering III:
Introduction to Embedded Systems
Spring 2023

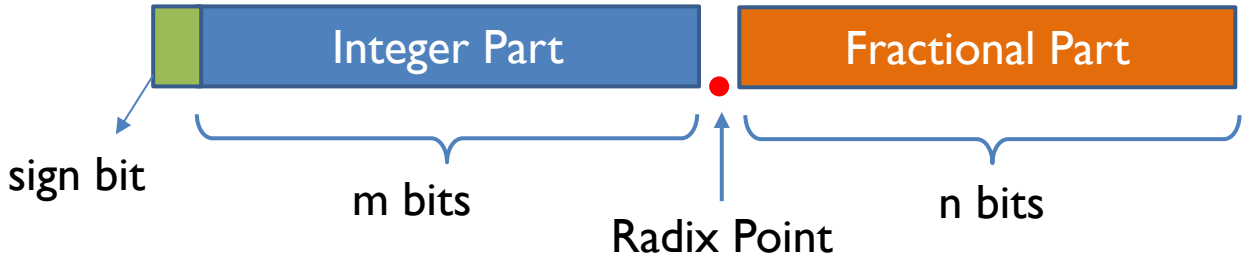
Taken from Dr. Yifeng Zhu's Lecture Notes
Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C – Chapter 12

Fixed-point Format: Q Notation

UQm.n for unsigned fixed-point

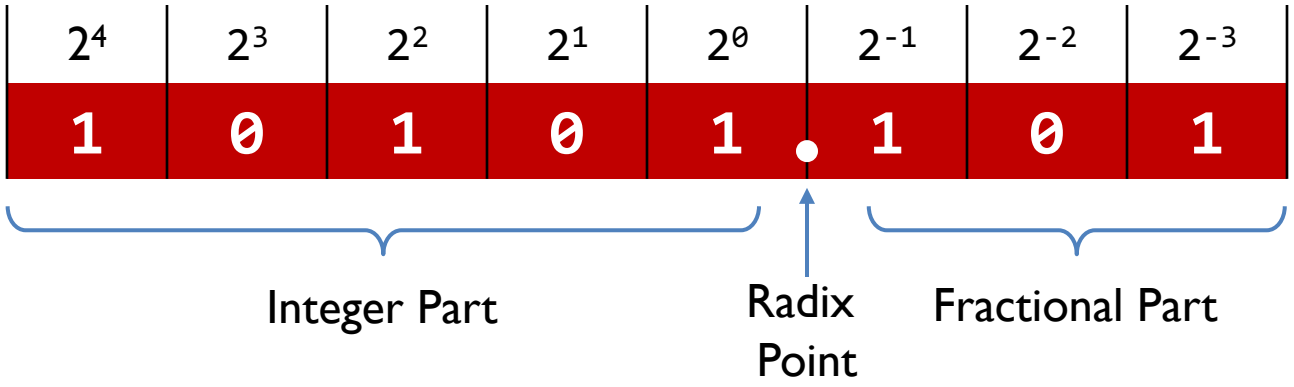


Qm.n for signed fixed-point



m + n + 1 bits

Converting Fixed-point UQ5.3 to Float



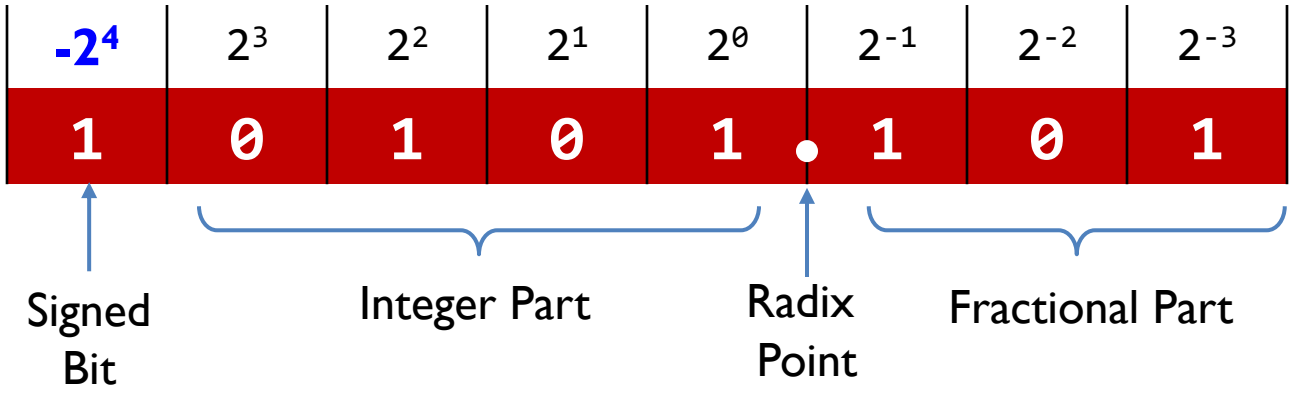
$\mathbf{10101.101}_2$

$= \mathbf{1}\times 2^4 + \mathbf{0}\times 2^3 + \mathbf{1}\times 2^2 + \mathbf{0}\times 2^1 + \mathbf{1}\times 2^0 + \mathbf{1}\times 2^{-1} + \mathbf{0}\times 2^{-2} + \mathbf{1}\times 2^{-3}$

$= 21.625$

$$\mathbf{10101.101}_2 = \frac{10101101_2}{2^3} = \frac{173}{8} = 21.625$$

Converting Fixed-point Q4.3 to Float



$$\mathbf{10101.101}_2$$

$$= \mathbf{1}\times(\mathbf{-2^4}) + \mathbf{0}\times 2^3 + \mathbf{1}\times 2^2 + \mathbf{0}\times 2^1 + \mathbf{1}\times 2^0 + \mathbf{1}\times 2^{-1} + \mathbf{0}\times 2^{-2} + \mathbf{1}\times 2^{-3}$$

$$= -10.375$$

$$\mathbf{10101.101}_2 = \frac{10101101_2}{2^3} = \frac{-83}{8} = -10.375$$

Convert Float to Fixed-point UQ4.12

UQm.n for unsigned fixed-point

$$Representation = round(float \times 2^n)$$

Example: Convert $f = 3.141593$ to UQ4.12

- Calculate $f \times 2^{12} = 12867.964928$
- Round the result to the nearest, $round(12867.964928) = 12868$
- Convert the integer to binary: 12868 = **11_0010_0100_0100**$_2$
- Organize into UQ4.12: **0011.0010_0100_0100**$_2$
- Final result in Hex: **0x3244**
- Error: $\frac{12868}{2^{12}} - f = -8.5625 \times 10^{-6}$

Convert Float to Fixed-point Q3.12

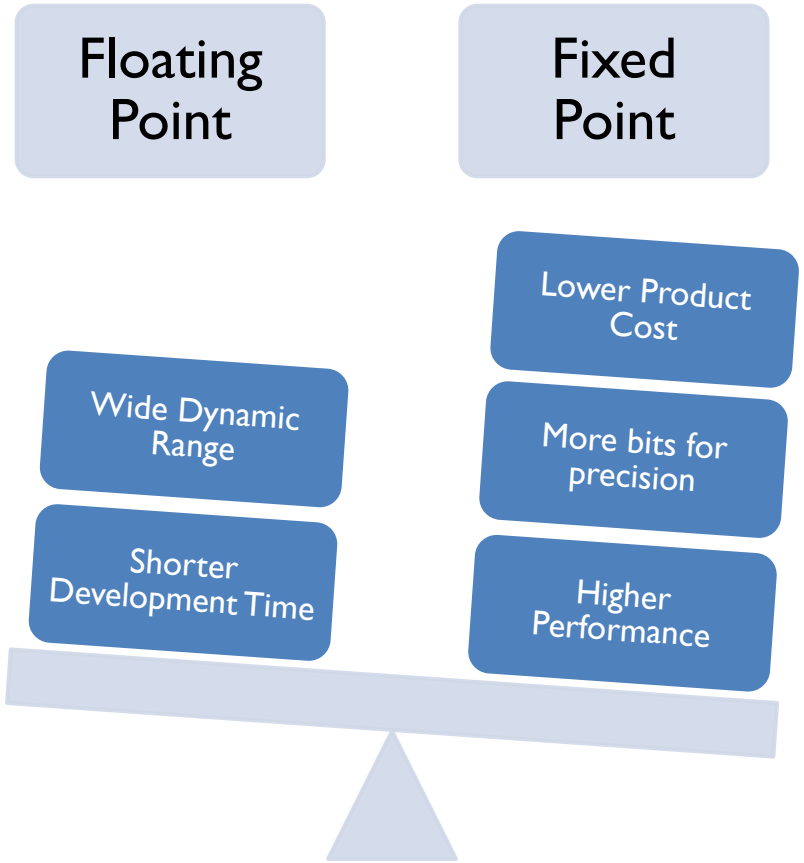
Qm.n for signed fixed-point

$$Representation = round(float \times 2^n)$$

Example: Convert $f = -3.141593$ to Q3.12

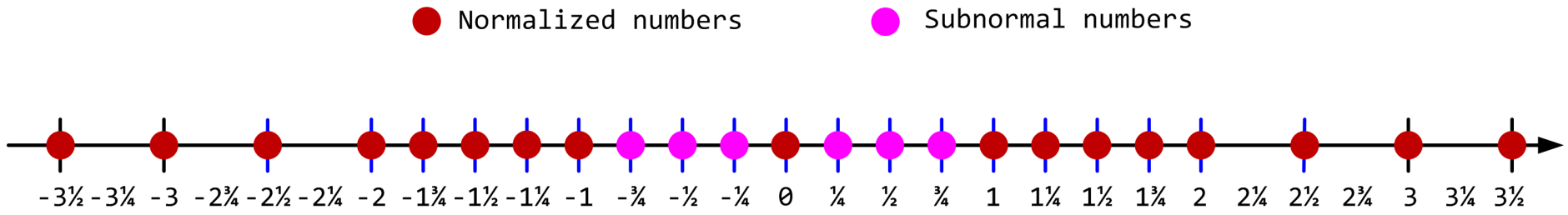
- Calculate $f \times 2^{12} = -12867.964928$
- Round the result to an integer, $round(-12867.964928) = -12868$
- Find the 16-bit two's complement: **1100_1101_1011_1100**$_2$
- Organize into Q3.12: **1100.1101_1011_1100**$_2$
- Final result in Hex: **0xCDBC**
- Error: $-\frac{12868}{2^{12}} - f = 8.5625 \times 10^{-6}$

Why use fixed-point?



Tradeoff between Range and Precision

A simplified 5-bit floating-point (IEEE 754 style)

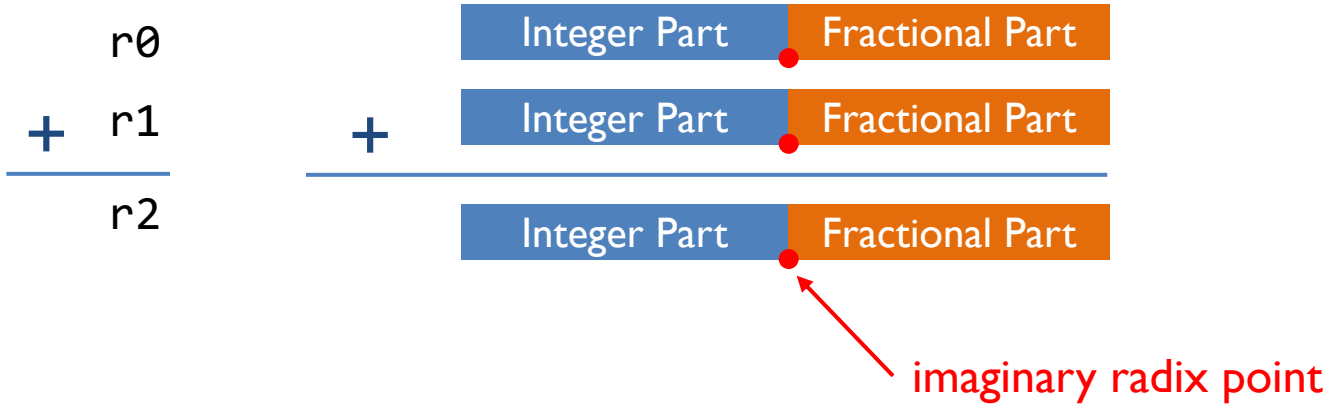


- Floating-Point
  - Resolution: difference between two neighbor numbers
  - Precision decreases as the magnitude increases
- Fixed-Point
  - Numbers are evenly distributed among the representable range
  - Precision is fixed

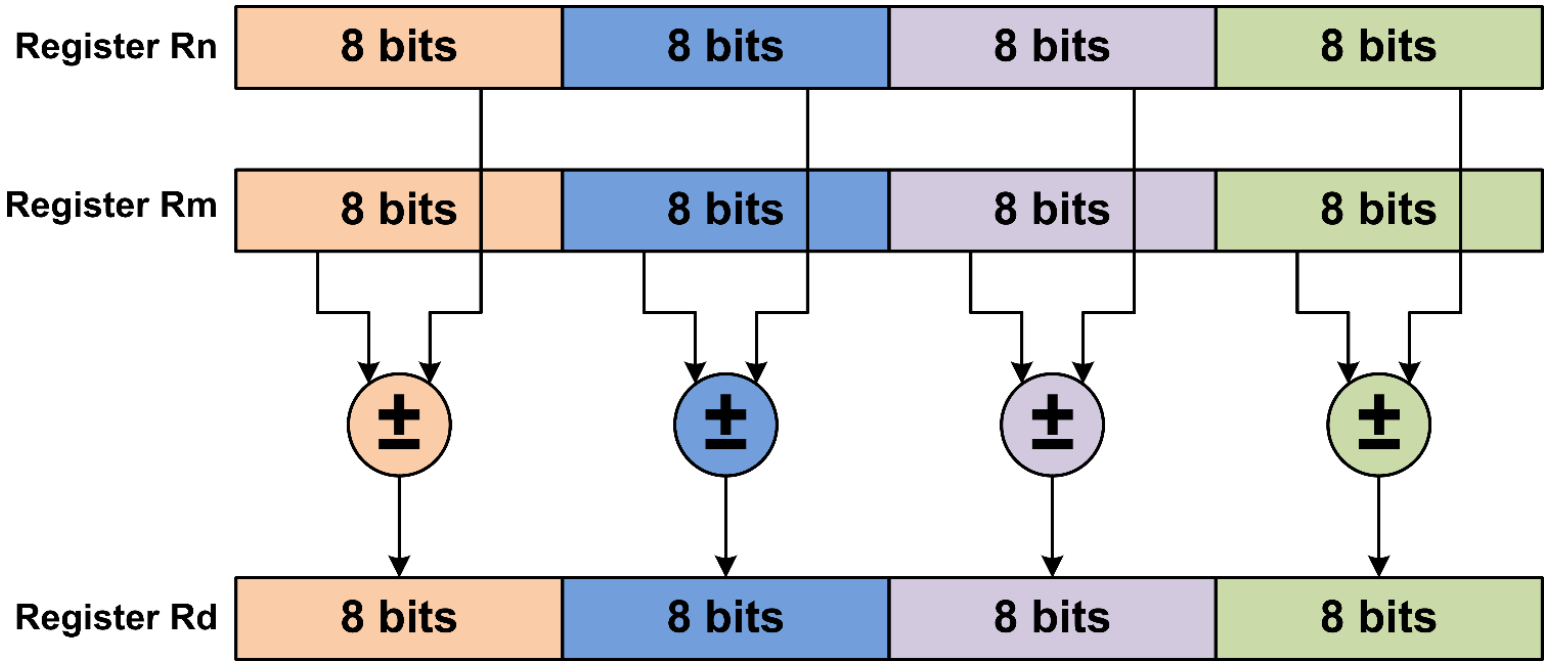
Fixed-point Addition and Subtraction

- Add/subtract two fixed-point numbers with the same format
  - Simply treat them as integers, and
  - Add/subtract using standard integer addition/subtraction

```
; r0 and r1 hold a UQ16.16
ADD r2, r0, r1    ; Simply use integer add
SUB r3, r0, r1    ; Simply use integer sub
```



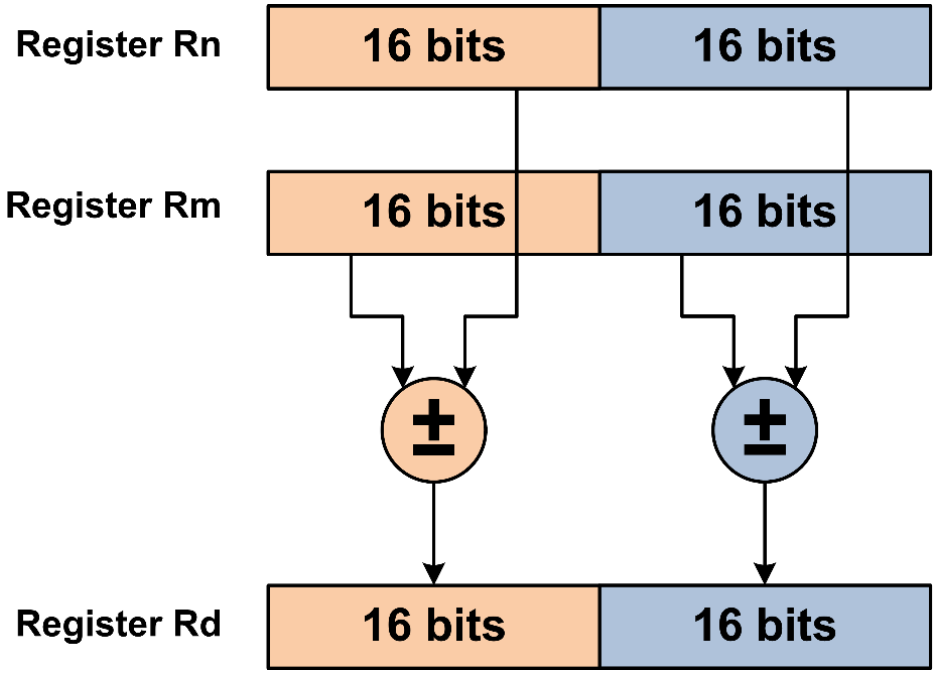
Parallel 8-bit Add and Subtract




```
SADD8   Rd, Rn, Rm
UADD8   Rd, Rn, Rm
```

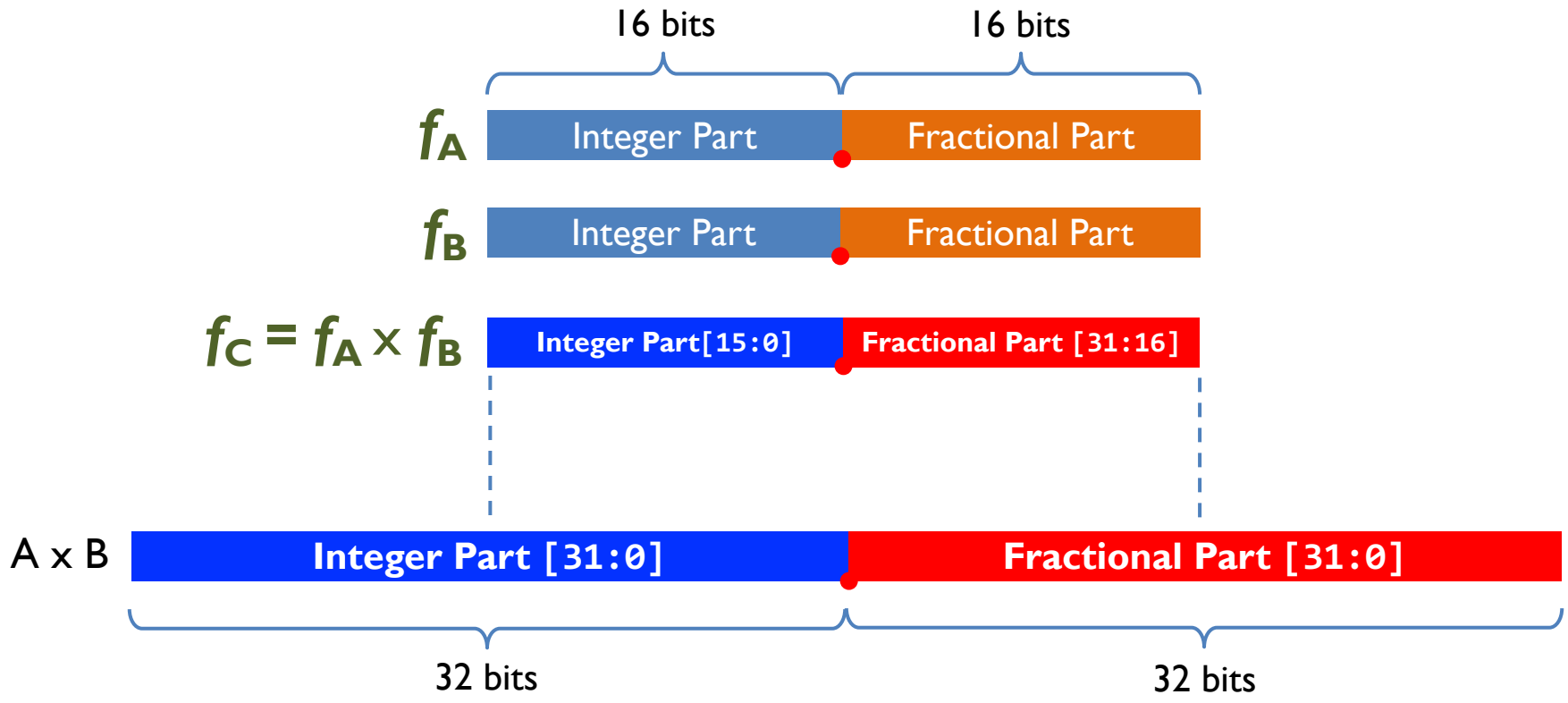
SIMD

Parallel 8-bit Add and Subtract

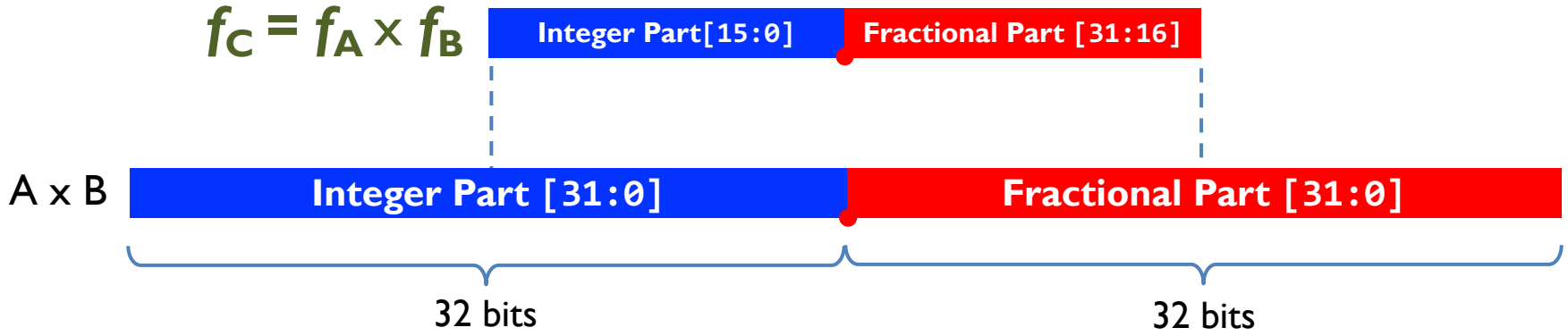



```
SADD16   Rd,Rn,Rm
UADD16   Rd,Rn,Rm
```

Fixed-point Multiplication in UQ16.16 or Q15.16



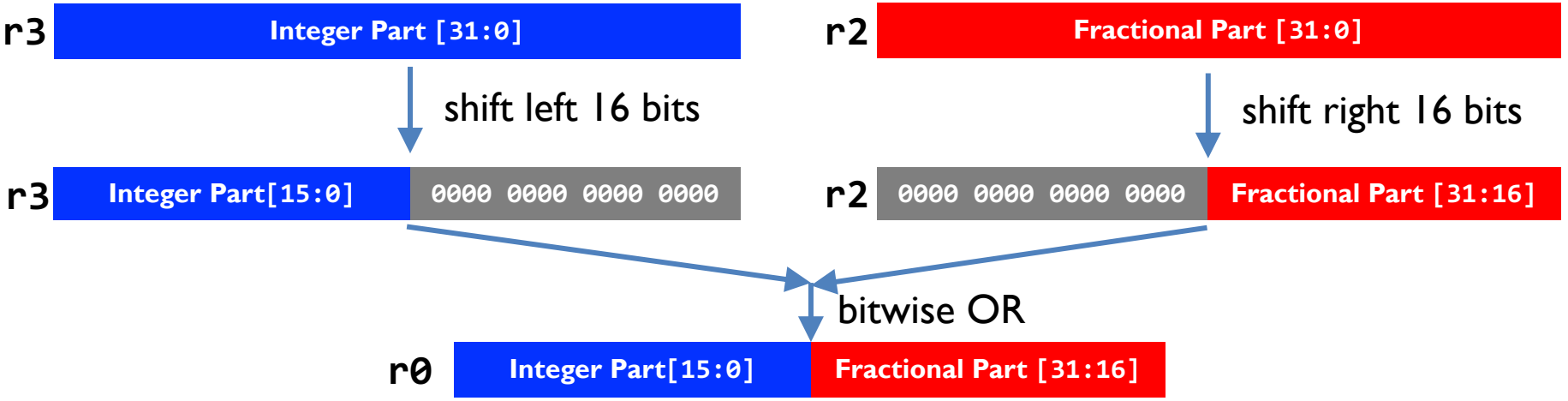
Fixed-point Multiplication in UQ16.16 or Q15.16



$$f_A = I_A \times 2^{-16} \qquad f_C = I_C \times 2^{-16}$$
$$f_B = I_B \times 2^{-16} \qquad f_C = f_A \times f_B$$

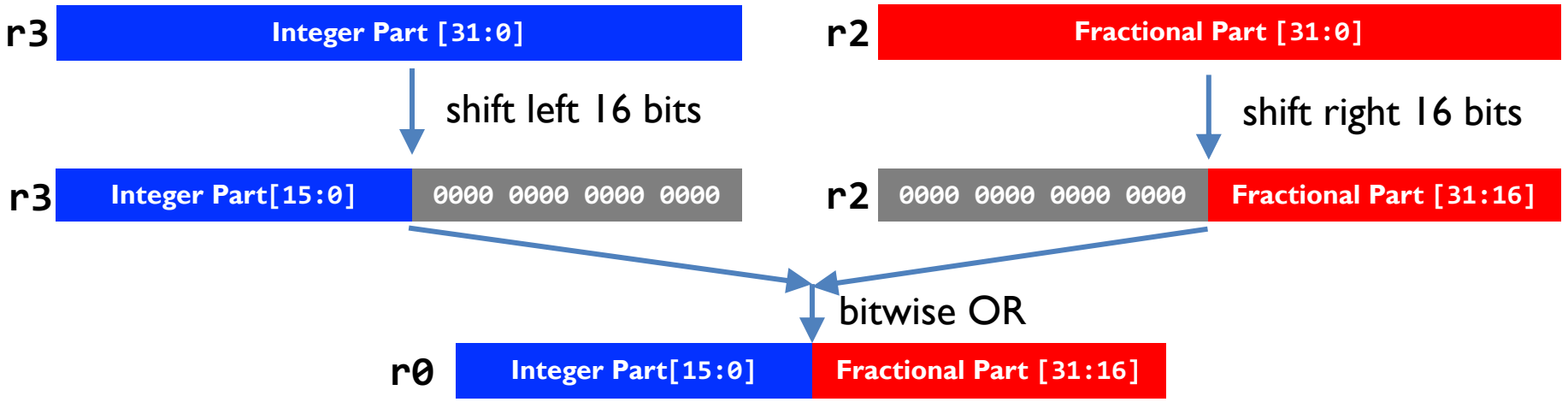
$$I_C = f_C \times 2^{16} = (f_A \times f_B) \times 2^{16} = (I_A \times 2^{-16}) \times (I_B \times 2^{-16}) \times 2^{16}$$
$$= (I_A \times I_B) \times 2^{-16}$$

Fixed-point Multiplication in UQ16.16



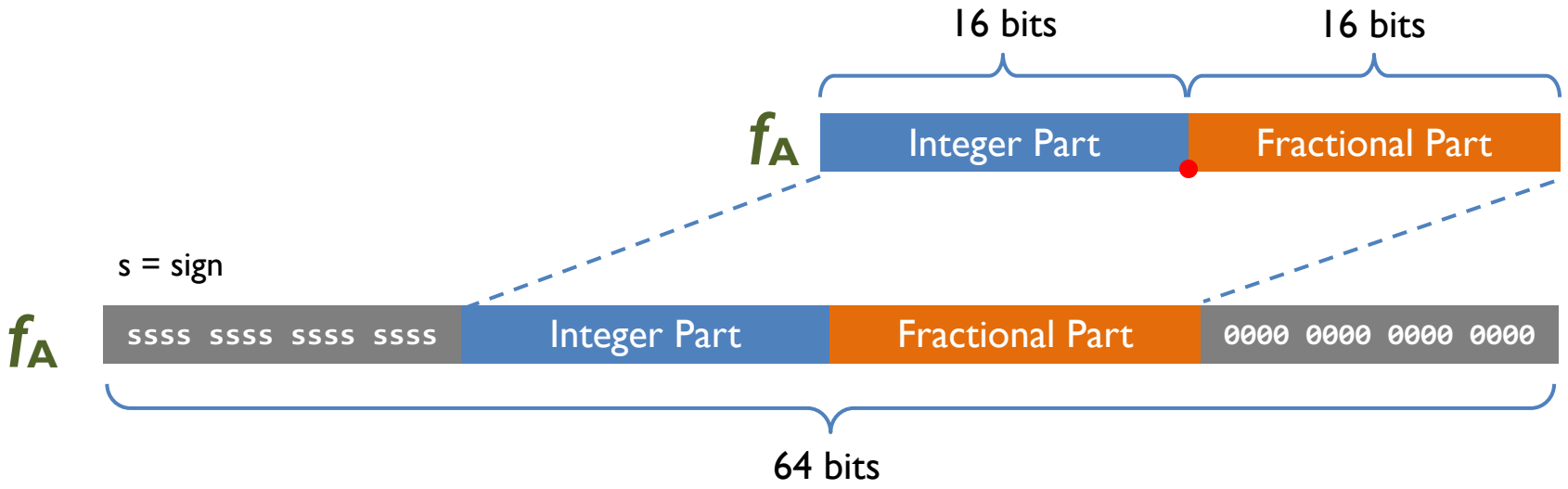

```
MUL_UQ16xUQ16 PROC
    UMULL r2, r3, r0, r1 ; Unsigned long multiply, 64-bit result in r3:r2
    LSLS  r3, r3, #16    ; Shift left the upper word r3
    LSRS  r2, r2, #16    ; Shift right the lower word r2
    ORR   r0, r2, r3     ; Pack two halfwords into a word
    BX    LR             ; Return
    ENDP
```

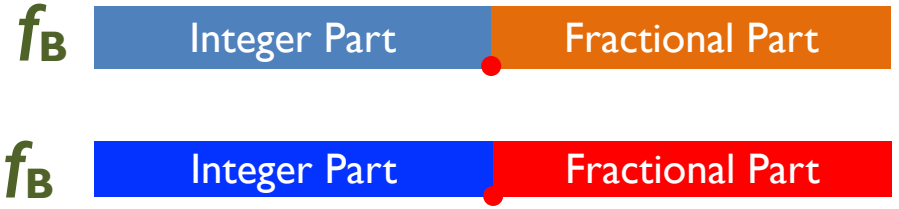
Fixed-point Multiplication in Q15.16




```
MUL_Q15xQ16 PROC
    SMULL r2, r3, r0, r1 ; Signed long multiply, 64-bit result in r3:r2
    LSLS  r3, r3, #16    ; Shift left the upper word r3
    LSRS  r2, r2, #16    ; Shift right the lower word r2
    ORR   r0, r2, r3     ; Pack two halfwords into a word
    BX    LR
    ENDP
```

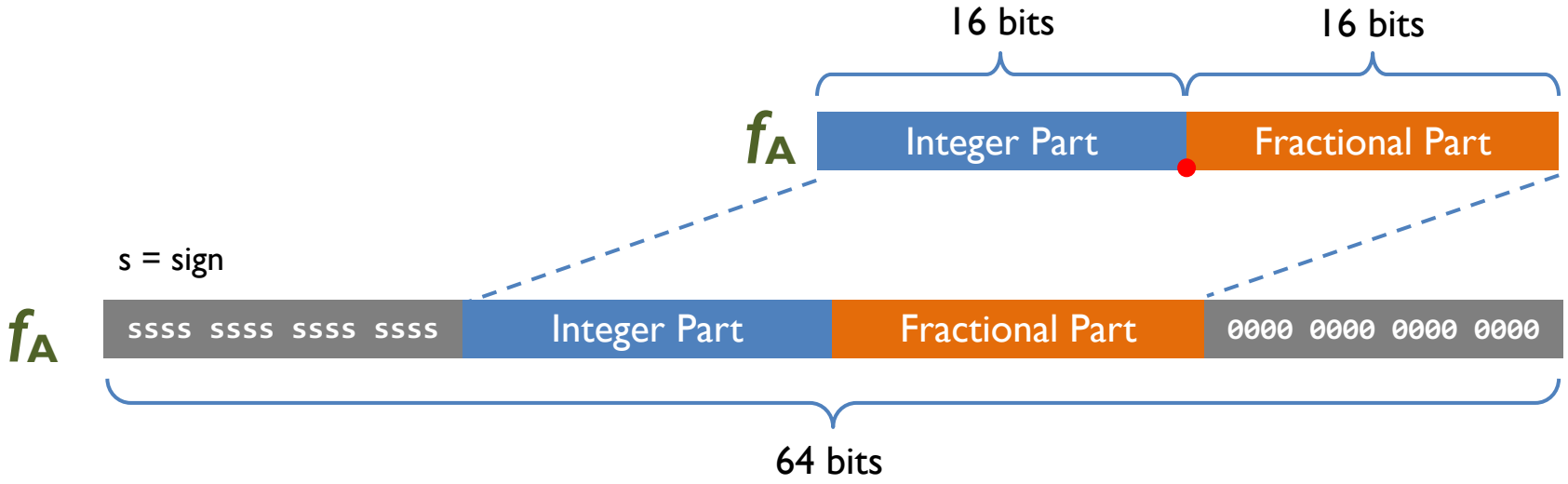
Fixed-point Division in UQ16.16





$$I_C = \frac{I_A \times 2^{16}}{I_B} = \frac{64\ bits}{32\ bits}$$

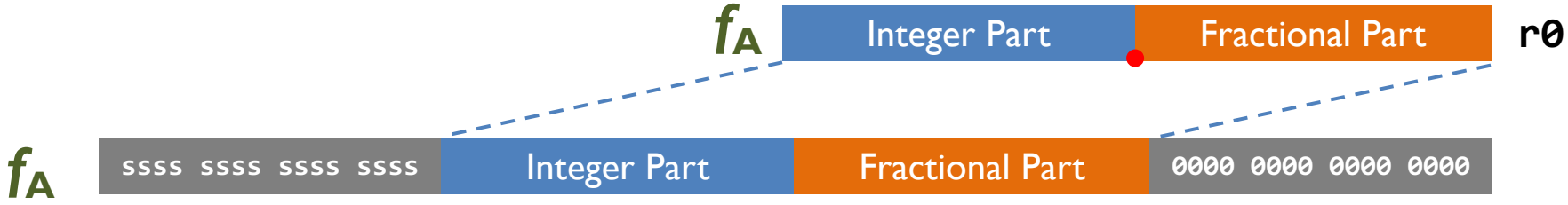
Fixed-point Division in UQ16.16



$$f_A = I_A \times 2^{-16} \qquad f_C = I_C \times 2^{-16}$$
$$f_B = I_B \times 2^{-16} \qquad f_C = f_A \div f_B$$

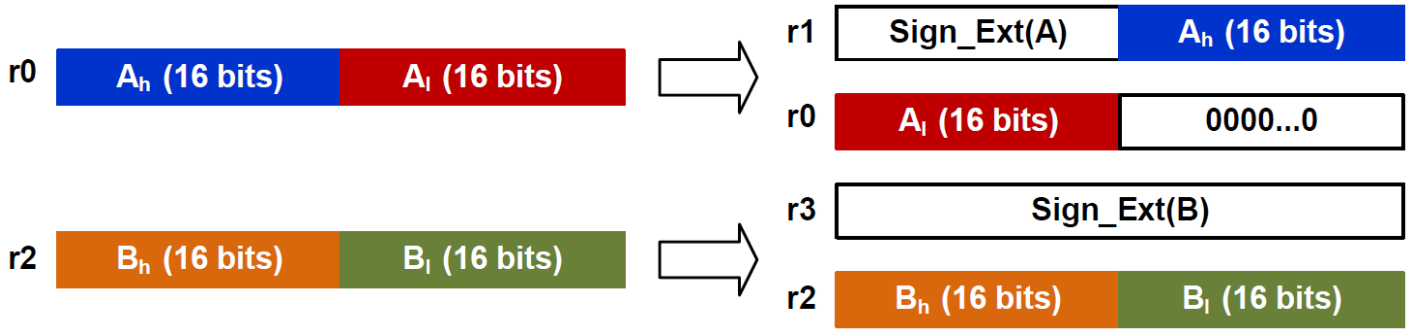
$$I_C = f_C \times 2^{16} = \frac{f_A}{f_B} \times 2^{16} = \frac{I_A \times 2^{-16}}{I_B \times 2^{-16}} \times 2^{16} = (I_A \times 2^{16}) \div I_B$$

Fixed-point Division in UQ16.16 or Q15.16




```
DIV_UQ16xUQ16 PROC
    PUSH {LR}
    ASRS   r1, r0, #31          ; r1[31:0] = sign bits of A
    LSLS   r1, r1, #16          ; put sign bits in upper half word
    ORR    r1, r1, r0, LSR #16
    LSLS   r0, r0, #16          ; [r1:r0] stores sign extended A
    ASRS   r3, r2, #31          ; [r3:r2] stores sign extended B
    BL     division_64_bits     ; Take four input register (r0:r1, r2:r3)
                                ; division_64_bits places result in r0
    POP    {pc}
    ENDP
```

Fixed-point Division in UQ16.16 or Q15.16



r4 = fixed-point quotient = A ÷ B

```
ASRS   r1, r0, #31          ; r1[31:0] = sign bits of A
LSLS   r1, r1, #16          ; put sign bits in upper half word
ORR    r1, r1, r0, LSR #16
LSLS   r0, r0, #16          ; [r1:r0] stores sign extended A
ASRS   r3, r2, #31          ; [r3:r2] stores sign extended B
BL     division_64_bits     ; Take four input register (r0:r1, r3:r2)
MOV    r4, r0               ; division_64_bits places result in r0
```